



Week 4: Proofs and Generics

CS-214 Software Construction

Outline

1. Proving Program Properties: Models, Induction, IntList
2. Proving properties on trees: IntTree
3. Automated proof checking
4. Parametric polymorphism, example of list and getting an element
5. Higher-order list functions defined on toy list
6. Scala lists: Construction, Patterns, Operations, isort
7. Tuples and generic methods, merge sort

Proving Program Properties

Proof by Induction over Integers

Say we wish to prove that some property, $P(n)$, holds for all integers starting from 0:



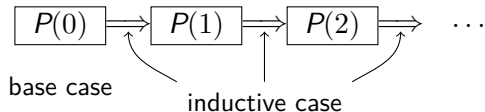
We need to show all of the above infinitely many cases of $P(n)$.

We start by proving $P(0)$. This is the *base case*.

Often property becomes harder the larger n is. Our strategy is, therefore, to show that the property *extends* from smaller to larger n . We prove:

Given any $n \geq 0$, if $P(n)$ then also $P(n+1)$. This is the *inductive case*.

Say we have shown base case, $P(0)$, and inductive case. Then:



Inductive Proof about a Function

```
def sum(n: Int): Int =  
  if n == 0 then 1  
  else n + sum(n - 1)
```

Prove: for every $n \geq 0$, it holds that: $\text{sum}(n) = n*(n+1)/2 + 1$

Base case: $n = 0$ Use substitution model to expand sum on the left side:

```
sum(0) = if 0 == 0 then 1  
        else 0 + sum(0 - 1)  
        = 1
```

Use evaluation of arithmetic operations for the right side:

$$0*(0+1)/2 + 1 = 0*1/2 + 1 = 0/2 + 1 = 0 + 1 = 1$$

Induction Step of $\text{sum}(n) = n*(n+1)/2 + 1$

Induction step: $n+1$ (where $n \geq 0$):

Assume Inductive Hypothesis (IH): $\text{sum}(n) = n*(n+1)/2 + 1$

```
sum(n + 1)
= if n + 1 == 0 then 1
  else n + 1 + sum((n + 1) - 1) // substitution model
= if false then 1                // (n+1 == 0) is false
  else n + 1 + sum(n + 1 - 1)
= n + 1 + sum(n + 1 - 1)        // if false
= n + 1 + sum(n)                 // cancel '+ 1 - 1'
= n + 1 + n*(n+1)/2 + 1          // Inductive Hypothesis
= (n+1)*((n+1)+1)/2 + 1         // arithmetic
```

`def sum(n: Int): Int =`
 `if n == 0 then 1`
 `else n + sum(n - 1)`

which proves $P(n) \Rightarrow P(n+1)$

Functions in Programs vs Math

Calling `sum` for large values of `n` may lead to outcomes for which proof does not apply:

- ▶ StackOverflow error. Stack is a data structure used to keep track of values of different recursive invocations of a function, but (especially in Java/Scala, has limited size), so deeply nested recursive calls result in error during execution.
- ▶ integer overflow: `Int.MaxValue == 2147483647` but `Int.MaxValue + 1 == -2147483648`

Possibly approaches for correct reasoning:

- ▶ use a precise model of machine integers: map results of arithmetic operations to $[-2^{31}, 2^{31} - 1]$ modulo 2^{32}
- ▶ check that values are not too large, then the result is the same as in math
- ▶ use unbounded integers for computation (as in Python or Haskell)

Unbounded Integers: BigInt

```
def sum(n: BigInt): BigInt = // using BigInt instead of Int as type
  if n == 0 then 1
  else n + sum(n - 1)
```

```
scala> val x: BigInt = 2147483647
```

```
val x: BigInt = 2147483647
```

```
scala> x + 1
```

```
val res0: BigInt = 2147483648
```

```
scala> x*x
```

```
val res1: BigInt = 4611686014132420609
```

```
scala> BigInt("7" * 60) + 1
```

[illegible]

Exercise with sumAcc

Test the following function with large values of $n \geq 0$:

```
def sumAcc(s: BigInt, n: BigInt): BigInt = {  
  if n == 0 then s  
  else sumAcc(s + n, n - 1)  
}
```

- ▶ Are you obtaining StackOverflow?
- ▶ Are you observing integer overflows?
- ▶ Propose a specification for the function and prove it by induction.
- ▶ Replace BigInt with Int, then find n such that `sumAcc(1, n)` returns a negative Int.

From Numbers to Lists

Lists are a generalization of non-negative integers. Take a list of Int-s.

```
sealed trait IntList
case object Nil extends IntList
case class Cons(head: Int, tail: IntList) extends IntList
```

For each non-negative integer n , there are (often many) lists of length n .

```
extension (xs: IntList)
  def length: BigInt =
    xs match
      case Nil => 0
      case Cons(h, t) => 1 + t.length
```

Concatenation (++) of Lists

```
extension (xs: IntList)
  def ++(ys: IntList): IntList =
    xs match
      case Nil => ys
      case Cons(h, t) => Cons(h, t ++ ys)
```

Example:

```
val l1 = Cons(1, Cons(2, Nil))
val l2 = Cons(10, Cons(20, Cons(30, Nil)))
l1 ++ l2 // Cons(1,Cons(2,Cons(10,Cons(20,Cons(30,Nil)))))
```

Concatenation (++) of Lists

```
extension (xs: IntList)
  def ++(ys: IntList): IntList =
    xs match
      case Nil => ys
      case Cons(h, t) => Cons(h, t ++ ys)
```

Example:

```
val l1 = Cons(1, Cons(2, Nil))
val l2 = Cons(10, Cons(20, Cons(30, Nil)))
l1 ++ l2 // Cons(1,Cons(2,Cons(10,Cons(20,Cons(30,Nil)))))
```

As a mathematical operation on sequences, which laws does ++ satisfy?

Concatenation (++) of Lists

```
extension (xs: IntList)
  def ++(ys: IntList): IntList =
    xs match
      case Nil => ys
      case Cons(h, t) => Cons(h, t ++ ys)
```

Example:

```
val l1 = Cons(1, Cons(2, Nil))
val l2 = Cons(10, Cons(20, Cons(30, Nil)))
l1 ++ l2 // Cons(1,Cons(2,Cons(10,Cons(20,Cons(30,Nil)))))
```

As a mathematical operation on sequences, which laws does ++ satisfy?

A small shorthand: $x :: xs$ abbreviates `Cons(x, xs)`

Laws of ++ That Hold by Definition

Two cases in the pattern match

```
extension (xs: IntList)
  def ++(ys: IntList): IntList =
    xs match
      case Nil => ys                // Nil case
      case Cons(h, t) => Cons(h, t ++ ys)  // Cons case
```

give us a two mathematical equations that hold by definition:

```
Nil ++ ys = ys                // Nil case - Nil is a left-neutral elem.
(h :: t) ++ ys == h :: (t ++ ys)  // Cons case
```

Associativity and Neutral Element of ++

We would like to *prove* the following additional laws:

```
xs ++ Nil    =  xs                // Nil is right-neutral element
(xs ++ ys) ++ zs = xs ++ (ys ++ zs) // concatenation is associative
```

We will use *structural induction* on lists.

This principle is analogous to induction on non-negative integers.

Structural Induction on Lists

To prove a property $P(xs)$ for all lists xs ,

- ▶ show that $P(\text{Nil})$ holds (*base case*),
- ▶ for an *arbitrary list* xs and an *arbitrary element* x , show the *induction step*:
if $P(xs)$ holds, then $P(x :: xs)$ also holds.

Consider any list $x_n :: (x_{n-1} \dots :: (x_1 :: \text{Nil}) \dots)$. Then

$$P(\text{Nil}) \Rightarrow P(x_1 :: \text{Nil}) \Rightarrow P(x_2 :: (x_1 :: \text{Nil})) \Rightarrow \dots \Rightarrow P(x_n :: (x_{n-1} :: \dots :: (x_1 :: \text{Nil}) \dots))$$

Each implication holds by the induction step.

The first step holds by base case, so the conclusion follows.

It also follows from the fact that each list has a finite non-negative length; we can do induction on this length.

Example Proof 1: Nil is Right-Neutral Element of ++

Remember defining equations:

```
Nil ++ ys = ys           // Nil case - Nil is a left-neutral elem.  
(h :: t) ++ ys == h :: (t ++ ys) // Cons case
```

We show by induction on xs that $xs ++ Nil = xs$

Base case:

```
Nil ++ Nil = Nil           // by Nil case
```

Induction Hypothesis: $xs ++ Nil = xs$

```
(x :: xs) ++ Nil  
= x :: (xs ++ Nil)         // by Cons case  
= x :: xs                  // by Induction Hypothesis
```

Example Proof 1: Nil is Right-Neutral Element of ++

Remember defining equations:

```
Nil ++ ys  =  ys           // Nil case - Nil is a left-neutral elem.  
(h :: t) ++ ys == h :: (t ++ ys)  // Cons case
```

We show by induction on xs that $xs ++ Nil = xs$

Base case:

```
Nil ++ Nil = Nil           // by Nil case
```

Induction Hypothesis: $xs ++ Nil = xs$

```
(x :: xs) ++ Nil  
= x :: (xs ++ Nil)         // by Cons case  
= x :: xs                  // by Induction Hypothesis
```

Example Proof 1: Nil is Right-Neutral Element of ++

Remember defining equations:

```
Nil ++ ys  =  ys           // Nil case - Nil is a left-neutral elem.  
(h :: t) ++ ys == h :: (t ++ ys)  // Cons case
```

We show by induction on xs that $xs ++ Nil = xs$

Base case:

```
Nil ++ Nil = Nil           // by Nil case
```

Induction Hypothesis: $xs ++ Nil = xs$

```
(x :: xs) ++ Nil  
= x :: (xs ++ Nil)         // by Cons case  
= x :: xs                  // by Induction Hypothesis
```

Example Proof 2: Associativity of ++

Let us now show that, for any three lists xs, ys, zs:

$$(xs \ ++ \ ys) \ ++ \ zs \ = \ xs \ ++ \ (ys \ ++ \ zs)$$

We again use the defining equations of ++

```
Nil ++ ys = ys           // Nil case - Nil is a left-neutral element
(h :: t) ++ ys == h :: (t ++ ys) // Cons case
```

We use structural induction on xs.

Base Case

Proving

$$(xs ++ ys) ++ zs = xs ++ (ys ++ zs)$$

Base case: xs is Nil

For the left-hand side we have:

$$\begin{aligned} & (Nil ++ ys) ++ zs \\ = & \quad ys ++ zs \quad \text{// by Nil case} \end{aligned}$$

Base Case

Proving

$$(xs ++ ys) ++ zs = xs ++ (ys ++ zs)$$

Base case: xs is Nil

For the left-hand side we have:

$$\begin{aligned} & (\text{Nil} ++ ys) ++ zs \\ &= ys ++ zs \quad // \text{ by Nil case} \end{aligned}$$

For the right-hand side, we have:

$$\begin{aligned} & \text{Nil} ++ (ys ++ zs) \\ &= ys ++ zs \quad // \text{ by Nil case, take ys in the case to be } (ys ++ zs) \end{aligned}$$

The base case is therefore proven.

Induction Step: Assume $(xs ++ ys) ++ zs = xs ++ (ys ++ zs)$

Induction step: first argument is $x :: xs$

For the left-hand side, we have:

```
((x :: xs) ++ ys) ++ zs
=  (x :: (xs ++ ys)) ++ zs    // by Cons case
=  x :: ((xs ++ ys) ++ zs)    // by Cons case
=  x :: (xs ++ (ys ++ zs))    // by Induction Hypothesis
```

For the right hand side we have:

```
(x :: xs) ++ (ys ++ zs)
=  x :: (xs ++ (ys ++ zs))    // by Cons case
```

Both left and right side are hence equal to

```
x :: (xs ++ (ys ++ zs))
```


Symbolic Execution

Let x, y be arbitrary values. Then using the same steps:

```
(x :: (y :: Nil)).length
= 1 + (y :: Nil).length      // Cons case
= 1 + (1 + Nil.length)      // Cons case
= 1 + (1 + 0)                // Nil case
= 2                           // arithmetic
```

We proved the following theorem: for every x, y

$$(x :: (y :: Nil)).length == 2$$

We can hence extend the substitution model to work not only with concrete inputs to functions, but also symbolic values and establish certain simple theorems. We call this *symbolic execution* because it is like executing the program, but without knowing all inputs (e.g. x, y).

Symbolic Execution

Let x, y be arbitrary values. Then using the same steps:

```
(x :: (y :: Nil)).length
= 1 + (y :: Nil).length      // Cons case
= 1 + (1 + Nil.length)      // Cons case
= 1 + (1 + 0)                // Nil case
= 2                          // arithmetic
```

We proved the following theorem: for every x, y

$$(x :: (y :: Nil)).length == 2$$

We can hence extend the substitution model to work not only with concrete inputs to functions, but also symbolic values and establish certain simple theorems. We call this *symbolic execution* because it is like executing the program, but without knowing all inputs (e.g. x, y).

Summary: Examples of Valid Equations You Can Use in Proofs

1. the defining equations for our functions, like `length` (for each case)
2. reflexivity: $E = E$
3. symmetry: if $E = F$ then $F = E$
4. transitivity: if $E = F$ and $F = G$, then $E = G$ (chaining: $E = F = G$)
5. instantiation: if $A = B$ holds for all values of symbolic x , then
 $A[C/x] = B[C/x]$ where C is any expression denoting a value.
Example: if $(x :: (y :: \text{Nil})).\text{length} = 2$ then $(42 :: (y :: \text{Nil})).\text{length} = 2$
6. substituting E by its equal F in another equality: if $E = F$ and $A = B$ then also
 $A[F/E] = B[F/E]$.
Example: if $f(x) = x + 1$ and $f(x) + 5 = z$, then $x + 1 + 5 = z$
7. result of composing above rules using symbolic execution and other strategies
8. equations proven using induction, using the above for base and inductive step

Structural Induction on Trees

Structural Induction on Trees

Structural induction is not limited to lists; it applies to any tree structure.

The general induction principle is the following:

To prove a property $P(t)$ for all trees t of a certain type,

- ▶ show that $P(lf)$ holds for all leaves lf of a tree,
- ▶ for each type of internal node t with subtrees $s_1, \dots, s_n$, show that $P(s_1) \wedge \dots \wedge P(s_n)$ *implies* $P(t)$.

Example: IntSets

Define binary trees that store sets of integers, faster than linear search:

```
sealed trait IntSet
case object Leaf extends IntSet
case class Node(left: IntSet, elem: Int, right: IntSet) extends IntSet

extension (s: IntSet)
  def contains(x: Int): Boolean = // is an element in the set?
    s match
      case Leaf => false
      case Node(l, e, r) =>
        if x < e then l.contains(x)
        else if x > e then r.contains(x)
        else true
```

Inserting into InSet: Preserving the Order

```
extension (s: IntSet)
  def incl(x: Int): IntSet = // a new set extended with a given element
    s match
      case Leaf => Node(Leaf, x, Leaf)
      case Node(l, e, r) =>
        if x < e then Node(l.incl(x), e, r)
        else if x > e then Node(l, e, r.incl(x))
        else s
```

The Laws of IntSet

What does it mean to prove the correctness of this implementation?

One way to define and show the correctness of an implementation consists of proving the laws that it respects.

In the case of IntSet, among the interesting laws are the following:

For any set s , and elements x and y :

```
Leaf.contains(x)          = false
s.incl(x).contains(x)     = true           // included x is present
s.incl(x).contains(y)     = s.contains(y), if x != y
```

This will show that IntSet behaves as a set with `incl` as operation that inserts an element into the set, and `contains` as set membership relation.

The first law follows by symbolic execution. We prove the other two by induction.

Included Present: `s.incl(x).contains(x)`

Structural induction on `s`.

Base case: Leaf follows by symbolic execution:

```
Leaf.incl(x).contains(x)
=   Node(Leaf, x, Leaf).contains(x) // case Leaf of incl
=   true                           // case Node, x=e of contains
```

Induction step: `s = Node(l, e, r)`, assuming the inductive hypotheses:

```
l.incl(x).contains(x) = true
r.incl(x).contains(x) = true
```

We have three cases: $x < e$, $x > e$, $x = e$.

Included Present: $s.incl(x).contains(x)$, cases $x < e$, $x > e$

Induction step: $s = \text{Node}(l, e, r)$ **where** $x < e$

```
Node(l, e, r).incl(x).contains(x)
=   Node(l.incl(x), e, r).contains(x) // case Node of incl, x < e
=   l.incl(x).contains(x)             // case Node of contains, x < e
=   true                             // Inductive Hypothesis
```

Induction step: $s = \text{Node}(l, e, r)$ **where** $x > e$ is analogous:

```
Node(l, e, r).incl(x).contains(x)
=   Node(l, e, r.incl(x)).contains(x) // case Node of incl, x > e
=   r.incl(x).contains(x)             // case Node of contains, x > e
=   true                             // Inductive Hypothesis
```

Included is Present: $s.incl(x).contains(x) = \text{true}$, case $x = e$

Induction step: $s = \text{Node}(l, e, r)$, $x = e$

```
Node(l, e, r).incl(x).contains(x)
=   Node(l, x, r).contains(x)      // case Node of incl, x = e
=   true                           // case Node of contains, x = e
```

This completes the proof by structural induction that, for all trees s and elements x ,
 $s.incl(x).contains(x) = \text{true}$

Others Remain Present

We now wish to prove the second key property of `incl`: if $x \neq y$ then

$$s.incl(x).contains(y) = s.contains(y)$$

(Elements other than x remain in or out of the extended set, as in s itself.)

Proof is by structural induction on s . Assume that $x < y$

(The case $x > y$ is analogous.)

Base case: Leaf

```
Leaf.incl(y).contains(x)
=   Node(Leaf, y, Leaf).contains(x)    // case Leaf of incl
=   Leaf.contains(x)                   // case Node of contains,  $x < y$ 
```

Others Remain Present: Inductive Cases

Proving: $s.\text{incl}(x).\text{contains}(y) = s.\text{contains}(y)$ when $x < y$

Induction step: $s = \text{Node}(l, e, r)$. Inductive hypotheses:

$l.\text{incl}(x).\text{contains}(y) = l.\text{contains}(y)$

$r.\text{incl}(x).\text{contains}(y) = r.\text{contains}(y)$

We distinguish five cases:

1. $e = y$

2. $e = x$

3. $e < x < y$

4. $x < e < y$

5. $x < y < e$

First Two Cases

Induction step: $\text{Node}(l, e, r)$, $e = y$, $x < y$ (so $x < e$)

```
Node(l, e, r).incl(x).contains(y)
= Node(l.incl(x), e, r).contains(y)    // case Node of incl,  $x < e$ 
= true                                // case Node of contains,  $y=e$ 
= Node(l, e, r).contains(y)           // case Node of contains,  $y=e$ 
```

Induction step: $\text{Node}(l, e, r)$, $e = x$, $x < y$ (so $e < y$)

```
Node(l, e, r).incl(x).contains(y)
= Node(l, e, r).contains(y)           // case Node of incl,  $e=x$ 
```

Case $e < x < y$

Induction step: $\text{Node}(l, e, r)$, $e < x < y$

```
Node(l, e, r).incl(x).contains(y)
= Node(l, e, r.incl(x)).contains(y) // case Node of incl,  $e < x$ 
= r.incl(x).contains(y)             // case Node of contains,  $e < y$ 
= r.contains(y)                     // Induction Hypothesis
= Node(l, e, r).contains(y)         // case Node of contains,  $e < y$ 
```

Case $x < e < y$

Induction step: $\text{Node}(l, e, r)$, $x < e < y$

```
Node(l, e, r).incl(x).contains(y)
= Node(l.incl(x), e, r).contains(y) // case Node of incl,  $x < e$ 
= r.contains(y)                     // case Node of contains,  $e < y$ 
= Node(l, e, r).contains(y)         // case Node of contains,  $e < y$ 
```

Case $x < y < e$

Induction step: `Node(l, e, r)`, $x < y < e$

```
Node(l, e, r).incl(x).contains(y)
= Node(l.incl(x), e, r).contains(y) // case Node of incl,  $x < e$ 
= l.incl(x).contains(y)             // case Node of incl,  $x < e$ 
= l.contains(y)                     // Induction Hypothesis
= Node(l, e, r).contains(y)         // case Node of contains,  $y < e$ 
```

These are all the cases, so the proposition is established.

Automated Proof Checking and Search

Proof Checking (for more see course CS-320)

It is easy to make a mistake when doing long proofs with many cases.

Proof checkers are programs that examine proof steps and make sure that each step is valid according to rules of logic.

Proof checkers are a basis of *proof assistants* which can also do automated proof search using (user-defined) *tactic* programs.

- ▶ Lean proof assistant: <https://lean-lang.org/>
- ▶ Rocq prover, <https://rocq-prover.org/>
- ▶ Isabelle proof assistant: <https://isabelle.in.tum.de/>
- ▶ HOL prover: <https://hol-theorem-prover.org/>
- ▶ Lisa proof framework: <https://github.com/epfl-lara/lisa>

In exercises you will use a small Scala library that checks proofs such as the ones you saw today when you execute it. Internally, it relies on Lisa proof assistant.

Generics (Parametric Polymorphism)

Type Parameters

Instead of defining list only for Int as in IntList, we can define it in general.

For this, we use generics: type parameters.

Type parameters are written in square brackets, e.g. [T]

- ▶ correspond approximately to <T> of Java

We can now generalize IntList using a type parameter:

```
sealed trait List[T]
case class Nil[T]() extends List[T] // made Nil a class to make T known
case class Cons[T](head: T, tail: List[T]) extends List[T]
```

Many List Methods Do Not Care about Type

```
extension[T] (xs: List[T])  
  def length: BigInt =  
    xs match  
      case Nil() => 0  
      case Cons(h, t) => 1 + t.length  
  
  def ++(ys: List[T]): List[T] =  
    xs match  
      case Nil() => ys  
      case Cons(h, t) => Cons(h, t ++ ys)
```

Generic Functions

Like classes, functions can have type parameters.

```
def singleton[T](elem: T) = Cons[T](elem, Nil[T]())
```

We can then write:

```
singleton[Int](1)  
singleton[Boolean](true)
```

Scala compiler can often deduce the correct type parameters from the value arguments of a function call, so we can write simply:

```
singleton(1)  
singleton(true)
```

Exercise with Generics

Write a function `nth` that takes a list and an integer `n` and selects the `n`'th element of the list.

Elements are numbered from 0.

If index is outside the range from 0 up to the length of the list minus one, a `IndexOutOfBoundsException` should be thrown.

```
def nth[T](xs: List[T], n: Int): T =  
  
  xs match  
    case Nil() => throw new IndexOutOfBoundsException()  
    case Cons(h, t) =>  
      if n == 0 then h  
      else nth(t, n - 1)
```

Exercise with Generics

Write a function `nth` that takes a list and an integer `n` and selects the `n`'th element of the list.

Elements are numbered from 0.

If index is outside the range from 0 up to the length of the list minus one, a `IndexOutOfBoundsException` should be thrown.

```
def nth[T](xs: List[T], n: Int): T =  
  
  xs match  
    case Nil() => throw new IndexOutOfBoundsException()  
    case Cons(h, t) =>  
      if n == 0 then h  
      else nth(t, n - 1)
```

Types and Evaluation

Type parameters do not affect evaluation in Scala.

We can assume that all type parameters and type arguments are removed before evaluating the program.

This is also called *type erasure*.

Languages that use type erasure include Java, Scala, Haskell, ML, OCaml.

Some other languages keep the type parameters around at run time, for example, C#

Higher-Order List Functions

Recurring Patterns for Computations on Lists

Examples in labs and exercises have shown that functions on lists often have similar structures.

We can identify several recurring patterns, like,

- ▶ transforming each element in a list in a certain way,
- ▶ retrieving a list of all elements satisfying a criterion,
- ▶ combining the elements of a list using an operator.

Scala allows programmers to write generic functions that such patterns once using **polymorphic higher-order functions**.

Proper use of higher-order functions *may* make code clearer and shorter.

Applying a Function to Elements of a List

A common operation is to transform each element of a list and then return the list of results.

For example, to multiply each element of a list by the same factor, you could write:

```
def scaleList(xs: List[Double], factor: Double): List[Double] = xs match
  case Nil      => xs
  case y :: ys => y * factor :: scaleList(ys, factor)
```

Mapping

This scheme can be generalized to the method `map` of the `List` class. A simple way to define `map` is as follows:

```
extension [T](xs: List[T])  
  def map[U](f: T => U): List[U] = xs match  
    case Nil      => xs  
    case x :: xs => f(x) :: xs.map(f)
```

Using `map`, `scaleList` can be written more concisely.

```
def scaleList(xs: List[Double], factor: Double) =  
  xs.map(x => x * factor)
```

We Need Type Parameters to Define map

In the boid lab so far, we avoided type parameters.

This required us to duplicate functions.

For example, you can apply mapBoid to many functions `f` but they all must accept a Boid:

```
def mapBoid(f: Boid => Boid): BoidSequence =  
  this match  
    case BoidNil()           => BoidNil()  
    case BoidCons(head, tail) => BoidCons(f(head), tail.mapBoid(f))
```

Exercise

Consider a function to square each element of a list, and return the result. Complete the two following equivalent definitions of `squareList`.

```
def squareList(xs: List[Int]): List[Int] = xs match
  case Nil      => ???
  case y :: ys => ???
```

```
def squareList(xs: List[Int]): List[Int] =
  xs.map(???)
```

Exercise

Consider a function to square each element of a list, and return the result. Complete the two following equivalent definitions of `squareList`.

```
def squareList(xs: List[Int]): List[Int] = xs match
  case Nil      => Nil
  case y :: ys => y * y :: squareList(ys)
```

```
def squareList(xs: List[Int]): List[Int] =
  xs.map(x => x * x)
```

Filtering

Another common operation on lists is the selection of all elements satisfying a given condition. For example:

```
def posElems(xs: List[Int]): List[Int] = xs match
  case Nil      => xs
  case y :: ys => if y > 0 then y :: posElems(ys) else posElems(ys)
```

Filter

This pattern is generalized by the method `filter` of the `List` class:

```
extension [T](xs: List[T])  
  def filter(p: T => Boolean): List[T] = this match  
    case Nil      => this  
    case x :: xs => if p(x) then x :: xs.filter(p) else xs.filter(p)
```

Using `filter`, `posElems` can be written more concisely.

```
def posElems(xs: List[Int]): List[Int] =  
  xs.filter(x => x > 0)
```

Variations of Filter

Besides filter, there are also the following methods that extract sublists based on a predicate:

- `xs.filterNot(p)` Same as `xs.filter(x => !p(x))`; The list consisting of those elements of `xs` that do not satisfy the predicate `p`.
- `xs.partition(p)` Same as `(xs.filter(p), xs.filterNot(p))`, but computed in a single traversal of the list `xs`.
- `xs.takeWhile(p)` The longest prefix of list `xs` consisting of elements that all satisfy the predicate `p`.
- `xs.dropWhile(p)` The remainder of the list `xs` after any leading elements satisfying `p` have been removed.
- `xs.span(p)` Same as `(xs.takeWhile(p), xs.dropWhile(p))` but computed in a single traversal of the list `xs`.

Reduction of Lists

Another common operation on lists is to combine the elements of a list using a given operator.

For example:

<code>sum(List(x1, ..., xn))</code>	<code>=</code>	<code>0 + x1 + ... + xn</code>
<code>product(List(x1, ..., xn))</code>	<code>=</code>	<code>1 * x1 * ... * xn</code>

We can implement this with the usual recursive schema:

```
def sum(xs: List[Int]): Int = xs match
  case Nil      => 0
  case y :: ys => y + sum(ys)
```

ReduceLeft

This pattern can be abstracted out using the generic method `reduceLeft`:

`reduceLeft` inserts a given binary operator between adjacent elements of a list:

$$\text{List}(x_1, \dots, x_n).\text{reduceLeft}(\text{op}) = x_1.\text{op}(x_2). \dots .\text{op}(x_n)$$

Using `reduceLeft`, we can simplify:

```
def sum(xs: List[Int])      = (0 :: xs).reduceLeft((x, y) => x + y)
def product(xs: List[Int]) = (1 :: xs).reduceLeft((x, y) => x * y)
```

A Shorter Way to Write Functions

Instead of `((x, y) => x * y)`, one can also write shorter:

`( _ * _ )`

Every `_` represents a new parameter, going from left to right.

The parameters are defined at the next outer pair of parentheses (or the whole expression if there are no enclosing parentheses).

So, `sum` and `product` can also be expressed like this:

```
def sum(xs: List[Int])      = (0 :: xs).reduceLeft(_ + _)
def product(xs: List[Int]) = (1 :: xs).reduceLeft(_ * _)
```

FoldLeft

The function `reduceLeft` is defined in terms of a more general function, `foldLeft`.

`foldLeft` is like `reduceLeft` but takes an *accumulator*, `z`, as an additional parameter, which is returned when `foldLeft` is called on an empty list.

$$\text{List}(x_1, \dots, x_n).\text{foldLeft}(z)(\text{op}) = z.\text{op}(x_1).\text{op} \dots .\text{op}(x_n)$$

So, `sum` and `product` can also be defined as follows:

```
def sum(xs: List[Int])      = xs.foldLeft(0)(_ + _)
def product(xs: List[Int])  = xs.foldLeft(1)(_ * _)
```

Implementations of ReduceLeft and FoldLeft

foldLeft and reduceLeft can be implemented in class List as follows.

```
abstract class List[T]:  
  
  def reduceLeft(op: (T, T) => T): T = this match  
    case Nil      => throw IllegalOperationException("Nil.reduceLeft")  
    case x :: xs => xs.foldLeft(x)(op)  
  
  def foldLeft[U](z: U)(op: (U, T) => U): U = this match  
    case Nil      => z  
    case x :: xs => xs.foldLeft(op(z, x))(op)
```

FoldRight and ReduceRight

Applications of foldLeft and reduceLeft create computation trees that lean to the left.

They have two dual functions, foldRight and reduceRight, which produce trees which lean to the right, e.g.,

```
List(x1, x2, x3).reduceRight(op)    = x1.op(x2.op(x3))
```

```
List(x1, x2, x3).foldRight(z)(op)  = x1.op(x2.op(x3.op(z)))
```

Implementation of FoldRight and ReduceRight

They are defined as follows

```
def reduceRight(op: (T, T) => T): T = this match
  case Nil          => throw UnsupportedOperationException("Nil.reduceRight")
  case x :: Nil     => x
  case x :: xs      => op(x, xs.reduceRight(op))

def foldRight[U](z: U)(op: (T, U) => U): U = this match
  case Nil          => z
  case x :: xs      => op(x, xs.foldRight(z)(op))
```

Difference between FoldLeft and FoldRight

For operators that are associative and commutative, `foldLeft` and `foldRight` are equivalent (even though there may be a difference in efficiency).

But sometimes, only one of the two operators is appropriate.

Exercise

```
def foldLeft[U](z: U)(op: (U, T) => U): U = this match
  case Nil      => z
  case x :: xs => xs.foldLeft(op(z, x))(op)
```

```
def foldRight[U](z: U)(op: (T, U) => U): U = this match
  case Nil      => z
  case x :: xs => op(x, xs.foldRight(z)(op))
```

Here is another formulation of concat:

```
def concat[T](xs: List[T], ys: List[T]): List[T] =
  xs.foldRight(ys)(_ :: _)
```

Here, it isn't possible to replace foldRight by foldLeft. What would go wrong?

Exercise: Reversing Lists using foldLeft

Use foldLeft to come up with a function for reversing lists which has a linear cost:

```
def reverse[T](xs: List[T]): List[T] = xs.foldLeft(z?)(op?)
```

for some expressions z? and op?.

Hint: use examples of inputs and expected outputs.

Note: coming up with an implementation that satisfies a specification is the problem of *program synthesis* and there are algorithms and systems that do this (even without LLMs).

Exercise

Complete the following definitions of the basic functions `map` and `length` on lists, such that their implementation uses `foldRight`:

```
def mapUsingFold[T, U](xs: List[T], f: T => U): List[U] =  
  xs.foldRight(Nil)( ??? )
```

```
def lengthUsingFold[T](xs: List[T]): Int =  
  xs.foldRight(0)( ??? )
```

More on Scala Lists

More on Lists in Scala

The list is a fundamental data structure in functional programming. In Scala, Lists are defined in the standard library.

A list having $x_1, \dots, x_n$ as elements is written `List( $x_1, \dots, x_n$ )`

Example

```
val fruit  = List("apples", "oranges", "pears")
val nums   = List(1, 2, 3, 4)
val diag3  = List(List(1, 0, 0), List(0, 1, 0), List(0, 0, 1))
val empty  = List()
```

There are two important differences between lists and arrays.

- ▶ Lists are immutable — the elements of a list cannot be changed.
- ▶ Lists have efficient addition to the front, but accessing n -th element is $O(n)$

The List Type

The type of a list with elements of type `T` is written `scala.List[T]` or shorter just `List[T]`

Each element of such list has type `T`

Example

```
val fruit: List[String]    = List("apples", "oranges", "pears")
val nums : List[Int]       = List(1, 2, 3, 4)
val diag3: List[List[Int]] = List(List(1, 0, 0), List(0, 1, 0), List(0, 0, 1))
val empty: List[Nothing]   = List()
```

Constructors of Lists

All lists are constructed from:

- ▶ the empty list `Nil`, and
- ▶ the construction operation `::` (pronounced *cons*):
 `x :: xs` gives a new list with the first element `x`, followed by the elements of `xs`.

For example:

```
fruit = "apples" :: ("oranges" :: ("pears" :: Nil))  
nums  = 1 :: (2 :: (3 :: (4 :: Nil)))  
empty = Nil
```

Right Associativity

Convention: Operators ending in “:” associate to the right.

$A :: B :: C$ is interpreted as $A :: (B :: C)$.

We can thus omit the parentheses in the definition above.

Example

```
val nums = 1 :: 2 :: 3 :: 4 :: Nil
```

Operations on Lists

All operations on lists can be expressed in terms of the following three:

head the first element of the list

tail the list composed of all the elements except the first.

isEmpty 'true' if the list is empty, 'false' otherwise.

These operations are defined as methods of objects of type `List`. For example:

```
fruit.head      == "apples"
```

```
fruit.tail.head == "oranges"
```

```
diag3.head      == List(1, 0, 0)
```

```
empty.head      == throw NoSuchElementException("head of empty list")
```

Decomposing Lists using Patterns

```
myList match  
  case p =>
```

Patterns p look analogous to operations to construct lists:

<code>Nil</code>	The <code>Nil</code> constant
<code>p :: ps</code>	A pattern that matches a list with a head matching p and a tail matching ps .
<code>List(p1, ..., pn)</code>	same as <code>p1 :: ... :: pn :: Nil</code>

Example

<code>1 :: 2 :: xs</code>	Lists of that start with 1 and then 2
<code>x :: Nil</code>	Lists of length 1
<code>List(x)</code>	Same as <code>x :: Nil</code>
<code>List()</code>	The empty list, same as <code>Nil</code>
<code>List(2 :: xs)</code>	List that contains as only element a list starting with 2

Sorting Lists

Suppose we want to sort `List(7, 3, 9, 2)` in ascending order:

1. One way is to sort the tail, `List(3, 9, 2)` to obtain `List(2, 3, 9)`
2. Then, insert the head 7 in the right place to obtain `List(2, 3, 7, 9)`.

This idea describes *Insertion Sort* :

```
def isort(xs: List[Int]): List[Int] = xs match
  case List() => List()
  case y :: ys => insert(y, isort(ys))
```

Insertion into a Sorted List

```
def insert(x: Int, xs: List[Int]): List[Int] = xs match
  case List() => List(x)
  case y :: ys =>
    if x < y then x :: xs else y :: insert(x, ys)
```

What is the asymptotic worst-case complexity of insert relative to the length of the input list N ?

- o constant time: $O(1)$
- o linear time: $O(N)$
- o $O(N * \log(N))$
- o $O(N * N)$

Insertion into a Sorted List

```
def insert(x: Int, xs: List[Int]): List[Int] = xs match
  case List() => List(x)
  case y :: ys =>
    if x < y then x :: xs else y :: insert(x, ys)
```

What is the asymptotic worst-case complexity of insert relative to the length of the input list N ?

- o constant time: $O(1)$
- o linear time: $O(N)$
- o $O(N * \log(N))$
- o $O(N * N)$

Exercise

Complete the definition insertion sort by filling in the ???s in the definition below:

```
def insert(x: Int, xs: List[Int]): List[Int] = xs match
  case List() => List(x)
  case y :: ys =>
    if x < y then x :: xs else y :: insert(x, ys)
```

What is the asymptotic worst-case complexity of isort relative to the length of the input list N ?

- ☐ constant time: $O(1)$
- ☐ linear time: $O(N)$
- ☐ $O(N * \log(N))$
- ☒ $O(N * N)$

Merge Sort (including Pairs)

Sorting Lists Faster

Let's design a function to sort lists that is more efficient than insertion sort.

An elegant and asymptotically good algorithm for this is *merge sort*.

The idea is as follows:

If the list consists of zero or one elements, it is already sorted.

Otherwise,

- ▶ Separate the list into two lists, each $\sim 1/2$ the length
- ▶ Sort the two sub-lists.
- ▶ Merge the two sorted sub-lists into a single sorted list.

First MergeSort Implementation

Here is the implementation of that algorithm in Scala:

```
def msort(xs: List[Int]): List[Int] =  
  val n = xs.length / 2  
  if n == 0 then xs  
  else  
    def merge(xs: List[Int], ys: List[Int]) = ???  
    val (fst, snd) = xs.splitAt(n)  
    merge(msort(fst), msort(snd))
```

The SplitAt Function

The `splitAt` function on lists returns two sublists

- ▶ the elements up to the given index
- ▶ the elements from that index

The lists are returned in a *pair*.

Detour: Pair and Tuples

The pair consisting of x and y is written (x, y) in Scala.

Example

```
val pair = ("answer", 42)  > pair : (String, Int) = (answer, 42)
```

The type of `pair` above is `(String, Int)`.

Pairs can also be used as patterns:

```
val (label, value) = pair  > label: String = answer, value: Int = 42
```

This works analogously for tuples with more than two elements.

If p is a pair, we can get its first element with `p._1`, second by `p._2`

Definition of Merge

Here is a definition of the merge function:

```
def merge(xs: List[Int], ys: List[Int]) = (xs, ys) match
  case (Nil, ys) => ys
  case (xs, Nil) => xs
  case (x :: xs1, y :: ys1) =>
    if x < y then x :: merge(xs1, ys)
    else y :: merge(xs, ys1)
```

Making Sort More General

Problem: How to parameterize `msort` so that it can also be used for lists with elements other than `Int`?

```
def msort[T](xs: List[T]): List[T] = ???
```

does not work, because the comparison `<` in `merge` is not defined for arbitrary types `T`.

Idea: Parameterize `merge` with the necessary comparison function.

Parameterization of Sort

The most flexible design is to make the function `sort` polymorphic and to pass the comparison operation as an additional parameter:

```
def msort[T](xs: List[T], lessThan: (T, T) => Boolean) =  
  ...  
  merge(msort(fst)(lt), msort(snd))
```

The inner function `merge` then needs to be adapted as follows:

```
def merge[T](xs: List[T], ys: List[T]) = (xs, ys) match  
  ...  
  case (x :: xs1, y :: ys1) =>  
    if lessThan(x, y) then ...  
    else ...
```

Resulting Generic Merge Sort

```
def msort[T](xs: List[T], lessThan: (T,T) => Boolean): List[T] =  
  val n = xs.length / 2  
  if n == 0 then xs  
  else  
    def merge(xs: List[T], ys: List[T]): List[T] =  
      (xs, ys) match  
        case (Nil, ys) => ys  
        case (xs, Nil) => xs  
        case (x :: xs1, y :: ys1) =>  
          if lessThan(x, y) then x :: merge(xs1, ys)  
          else y :: merge(xs, ys1)  
  
    val (firstHalf, secondHalf) = xs.splitAt(n)  
    merge(msort(firstHalf, lessThan),  
          msort(secondHalf, lessThan))
```

Calling Parameterized Sort

We can now call `msort` as follows:

```
val xs = List(-5, 6, 3, 2, 7)
val fruits = List("apple", "pear", "orange", "pineapple")

msort(xs, (x: Int, y: Int) => x < y)
msort(fruits, (x: String, y: String) => x.compareTo(y) < 0)
```

Or, since parameter types can be inferred from the call `msort(xs)`:

```
msort(xs, (x, y) => x < y)
```